Name: Vansh Jain

Roll No: 124A8036

Class/Div: SE AIDS/G2

Experiment No. 09

Title: To implement and demonstrate demand paging and page replacement algorithms (FIFO, LRU, and Optimal) for simulating virtual memory management.

Objective:

- To understand the concept of virtual memory and its implementation using demand paging.
- To simulate demand paging and observe how pages are loaded into memory on demand.
- To study the occurrence of page faults and their impact on system performance.
- To implement different page replacement algorithms such as FIFO, LRU, and Optimal.
- To compare the performance of these algorithms based on the number of page faults.
- To analyze the efficiency of memory management techniques in operating systems.

Software: Linux operating system

Theory:

A. Demand Paging:

Demand Paging is a memory management technique used in operating systems where pages are loaded into main memory only when they are required (on demand), rather than loading the entire process at once.

In this scheme, when a program tries to access a page that is not currently in memory, a page fault occurs. The operating system then loads the required page from secondary storage (disk) into a free frame in main memory. If no free frame is available, a page replacement algorithm is used to replace an existing page.

Demand paging helps in:

- ✧ Reducing memory usage
- ✧ Increasing multiprogramming
- ✧ Improving system efficiency

However, excessive page faults can lead to performance degradation, known as thrashing.

B. Page Replacement Algorithms:

When a page fault occurs and no free frame is available, the system must decide which page to remove. This decision is made using page replacement algorithms.

1. FIFO (First-In First-Out) Algorithm

FIFO is the simplest page replacement algorithm. It replaces the page that has been in memory for the longest time.

- ✧ Pages are maintained in a queue.
- ✧ The oldest page (first entered) is removed first.

Advantages:

- ✧ Easy to implement
- ✧ Low overhead

Disadvantages:

- ✧ May remove frequently used pages
- ✧ Can suffer from Belady's Anomaly (more frames can increase page faults)

2. LRU (Least Recently Used) Algorithm

LRU replaces the page that has not been used for the longest period of time.

- ✧ It keeps track of usage history of pages.
- ✧ Assumes that pages used recently will be used again soon.

Advantages:

- ✧ Better performance than FIFO
- ✧ Reduces unnecessary page faults

Disadvantages:

- ✧ More complex to implement
- ✧ Requires additional tracking (counters or stack)

3. Optimal Page Replacement Algorithm

The Optimal algorithm replaces the page that will not be used for the longest time in the future.

- ✧ It looks ahead in the reference string to decide which page to replace.

Advantages:

- ✧ Produces the minimum number of page faults
- ✧ Used as a benchmark for comparison

Disadvantages:

- ✧ Not practically implementable (future knowledge required)

Output/Result:

A. Demand Paging:

```
  GNU nano 4.8                                                    demand_paging.c
#include <stdio.h>
#define MAX 50

int main()
{
    int pages[MAX], frames[MAX];
    int n, f;
    int pageFaults = 0;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    // Initialize frames with -1
    for(int i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nPage\tFrames\n");

    for(int i = 0; i < n; i++)
    {
        int page = pages[i];
        int found = 0;

        // Check if page is already in frame
        for(int j = 0; j < f; j++)
        {
            if(frames[j] == page)
            {
                found = 1;
                break;
            }
        }

        // If not found → Page Fault
        if(found == 0)
        {
            pageFaults++;

            // Find empty frame
            int placed = 0;
            for(int j = 0; j < f; j++)
            {
                if(frames[j] == -1)
                {
```

```
                if(frames[j] == -1)
                {
                    frames[j] = page;
                    placed = 1;
                    break;
                }
            }

            // If no empty frame → replace first (simple FIFO style)
            if(placed == 0)
            {
                for(int j = 0; j < f-1; j++)
                    frames[j] = frames[j+1];

                frames[f-1] = page;
            }
        }

        // Display frames
        printf("%d\t", page);
        for(int j = 0; j < f; j++)
        {
            if(frames[j] == -1)
                printf("- ");
            else
                printf("%d ", frames[j]);
        }
        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", pageFaults);

    return 0;
}
```

```
siesgst@puppet-master:/home/exp9$ sudo nano demand_paging.c
siesgst@puppet-master:/home/exp9$ sudo gcc demand_paging.c -o demand_paging
siesgst@puppet-master:/home/exp9$ ./demand_paging.c
bash: ./demand_paging.c: Permission denied
siesgst@puppet-master:/home/exp9$ ./demand_paging
Enter number of pages: 8
Enter page reference string:
1 2 3 4 1 2 5 1
Enter number of frames: 3

Page    Frames
1       1 - -
2       1 2 -
3       1 2 3
4       2 3 4
1       3 4 1
2       4 1 2
5       1 2 5
1       1 2 5

Total Page Faults = 7
```

B. Page Replacement Algorithms:

1. FIFO

```
  GNU nano 4.8                                                    fifo.c
#include <stdio.h>
#define MAX 50
int main()
{
    int pages[MAX], frames[MAX];
    int n, f, front = 0;
    int pageFaults = 0, pageHits = 0;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(int i = 0; i < f; i++)
        frames[i] = -1;
    printf("\nPage\tFrames\n");
    for(int i = 0; i < n; i++)
    {
        int found = 0;
        for(int j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                pageHits++;
                break;
            }
        }
        if(found == 0)
        {
            frames[front] = pages[i];
            front = (front + 1) % f;
            pageFaults++;
        }
        printf("%d\t", pages[i]);
        for(int j = 0; j < f; j++)
            printf("%d ", frames[j]);
        printf("\n");
    }

    printf("\nFIFO Page Faults = %d", pageFaults);
    printf("\nFIFO Page Hits = %d\n", pageHits);

    return 0;
}
```

```
siesgst@puppet-master:/home/exp9$ sudo nano fifo.c
siesgst@puppet-master:/home/exp9$ sudo gcc fifo.c -o fifo
siesgst@puppet-master:/home/exp9$ ./fifo
Enter number of pages: 12
Enter page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

Page    Frames
1       1 -1 -1
2       1 2 -1
3       1 2 3
4       4 2 3
1       4 1 3
2       4 1 2
5       5 1 2
1       5 1 2
2       5 1 2
3       5 3 2
4       5 3 4
5       5 3 4

FIFO Page Faults = 9
FIFO Page Hits = 3
```

2. LRU

```
  GNU nano 4.8                                                    lru.c
#include <stdio.h>
#define MAX 50
int main()
{
    int pages[MAX], frames[MAX], time[MAX];
    int n, f, counter = 0;
    int pageFaults = 0, pageHits = 0;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &f);
    for(int i = 0; i < f; i++)
    {
        frames[i] = -1;
        time[i] = 0;
    }
    printf("\nPage\tFrames\n");
    for(int i = 0; i < n; i++)
    {
        int found = 0;
        for(int j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                pageHits++;
                time[j] = counter++;
                break;
            }
        }
        if(found == 0)
        {
            int lru = 0;
            for(int j = 1; j < f; j++)
            {
                if(time[j] < time[lru])
                    lru = j;
            }
            frames[lru] = pages[i];
            time[lru] = counter++;
            pageFaults++;
        }
```

```
                found = 1;
                pageHits++;
                time[j] = counter++;
                break;
            }
        }
        if(found == 0)
        {
            int lru = 0;
            for(int j = 1; j < f; j++)
            {
                if(time[j] < time[lru])
                    lru = j;
            }
            frames[lru] = pages[i];
            time[lru] = counter++;
            pageFaults++;
        }
        printf("%d\t", pages[i]);
        for(int j = 0; j < f; j++)
            printf("%d ", frames[j]);
        printf("\n");
    }
    printf("\nLRU Page Faults = %d", pageFaults);
    printf("\nLRU Page Hits = %d\n", pageHits);
    return 0;
}
```

```
siesgst@puppet-master:/home/exp9$ sudo nano lru.c
siesgst@puppet-master:/home/exp9$ sudo gcc lru.c -o lru
siesgst@puppet-master:/home/exp9$ ./lru
Enter number of pages: 12
Enter page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

Page    Frames
1       1 -1 -1
2       2 -1 -1
3       2 3 -1
4       2 3 4
1       1 3 4
2       1 2 4
5       1 2 5
1       1 2 5
2       1 2 5
3       1 2 3
4       4 2 3
5       4 5 3

LRU Page Faults = 10
LRU Page Hits = 2
```

3. Optimal

```
  GNU nano 4.8                                                    optimal.c
#include <stdio.h>
#define MAX 50

int main()
{
    int pages[MAX], frames[MAX];
    int n, f;
    int pageFaults = 0, pageHits = 0;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(int i = 0; i < f; i++)
        frames[i] = -1;

    printf("\nPage\tFrames\n");

    for(int i = 0; i < n; i++)
    {
        int found = 0;

        for(int j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                pageHits++;
                break;
            }
        }
```

```
        if(found == 0)
        {
            int pos = -1, farthest = i;

            for(int j = 0; j < f; j++)
            {
                int k;
                for(k = i + 1; k < n; k++)
                {
                    if(frames[j] == pages[k])
                        break;
                }

                if(k > farthest)
                {
                    farthest = k;
                    pos = j;
                }
            }

            if(pos == -1)
                pos = 0;

            frames[pos] = pages[i];
            pageFaults++;
        }

        printf("%d\t", pages[i]);
        for(int j = 0; j < f; j++)
            printf("%d ", frames[j]);
        printf("\n");
    }

    printf("\nOptimal Page Faults = %d", pageFaults);
    printf("\nOptimal Page Hits = %d\n", pageHits);

    return 0;
}
```

```
siesgst@puppet-master:/home/exp9$ sudo nano optimal.c
siesgst@puppet-master:/home/exp9$ sudo gcc optimal.c -o optimal
siesgst@puppet-master:/home/exp9$ ./optimal
Enter number of pages: 12
Enter page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

Page    Frames
1       1 -1 -1
2       1 2 -1
3       1 2 3
4       1 2 4
1       1 2 4
2       1 2 4
5       1 2 5
1       1 2 5
2       1 2 5
3       3 2 5
4       4 2 5
5       4 2 5

Optimal Page Faults = 7
Optimal Page Hits = 5
```

Conclusion: The experiment demonstrated the working of demand paging and various page replacement algorithms in virtual memory management. It showed how different algorithms affect the number of page faults and overall system performance. Among them, the Optimal algorithm gives the best results, while FIFO and LRU vary in efficiency based on usage patterns.